3D Views for Medical Reports

Project Plan · CISC 3171: Introduction to Software Engineering

Version: 1.2 · **Updated:** Oct 29, 2026 · **Team:** The White Coats

1. What We're Doing

3D Views for Medical Reports is a desktop browser web application that visualizes broken bones in the human body. A person uploads an initial radiology imaging report (a PDF or pasted text (images upcoming)), confirms the fracture findings, and the app shows an interactive 3D view of a 126-bone adult human skeleton with the highlighted fractures. As the person recovers and receives more reports from their doctor, they can upload subsequent reports (after creating an account) to view a chronological recovery timeline for their fractures and compare reports.

Z-Anatomy, an open-source 3D anatomy atlas, provides the skeleton model and the inspiration for the interactive 3D scene.

For our MVP, we formally lock the release scope exclusively to Priority 1 (P1) features (F1–F6): adding reports, verifying the findings, the 3D skeleton, viewing both the report and model simultaneously, user accounts, and recovery timelines. All P2 features (F7–F11) are explicitly deferred to post-MVP stages and will only begin if remaining time permits (see Timeline). S7.3 (sorting reports by date or bone) is also left out of this plan, since F5 already covers most of it.

2. Why We're Doing It

Radiology imaging reports are written by radiologists for other physicians and contain verbose, clinical terminology, such as "nondisplaced fracture of the distal left radius." Patients often read these reports in their patient portals before the ordering physician reviews the reports with them. This leaves patients in the dark as they try to decipher the intimidating medical jargon on their own, often incorrectly.

Existing tools each cover one piece: 3D anatomy viewers such as Z-Anatomy and the Merck Manual skeleton show bones but not a patient's report, and patient portals show the report but no picture of it. The 2D visuals patients receive are only so helpful for depicting our 3D world; they don't show depth and sometimes contain overlap, which introduces ambiguity. We believe our team can bridge this gap, creating an application that analyzes the reports and visualizes them in a 3D space, so patients can be at ease while trying to understand their injuries. The Software Pitch has the full problem statement, and Requirements Document v2 defines the scope this plan builds on.

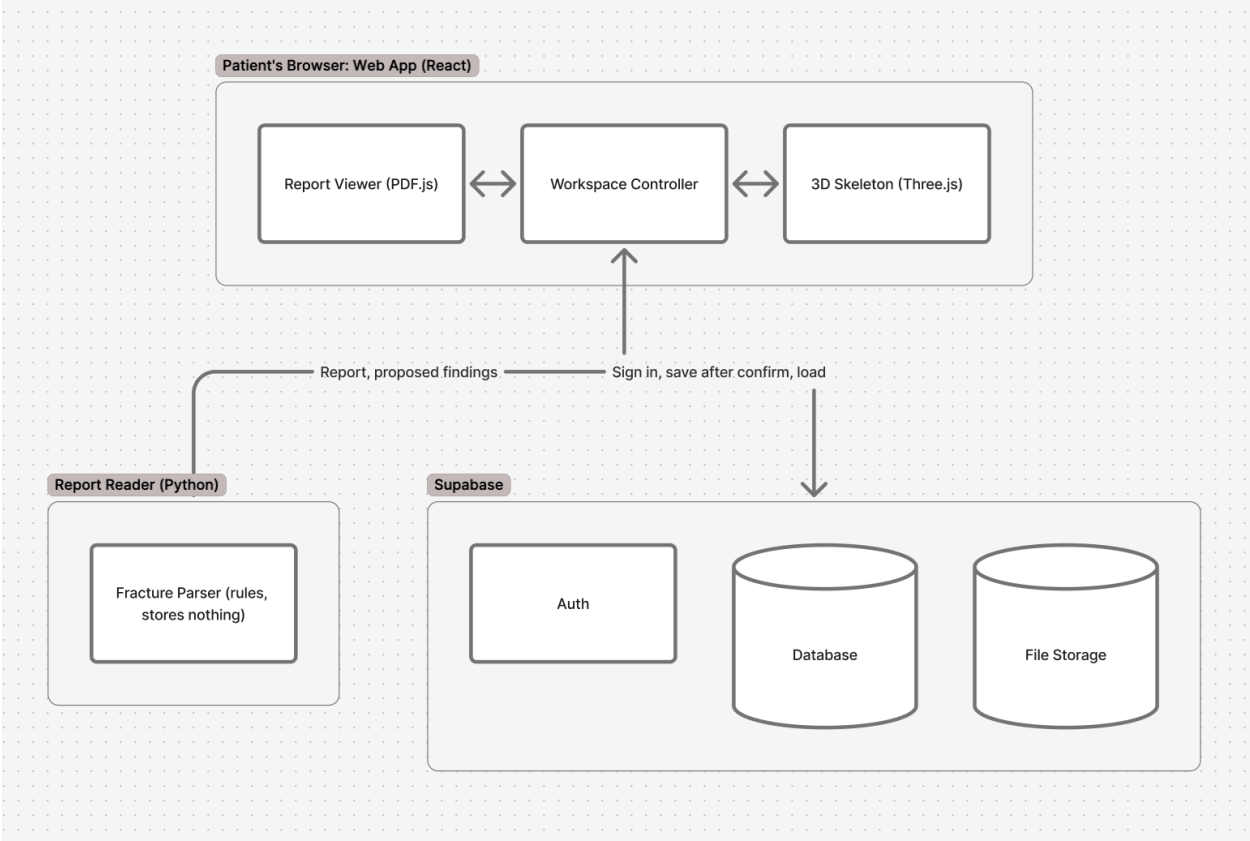
3. How We're Doing It

Design

Stack

- **React and TypeScript, built with Vite,** for the web app. Every screen is a React component, and TypeScript catches findings with missing fields before they reach the model.
- **Three.js** renders the 3D appendicular skeleton in a scene that allows for camera orbit/pan controls and other interactions (e.g., selecting particular bones, isolating them, etc.). It runs through **React Three Fiber and drei**: React Three Fiber lets the 3D scene and the findings list share the same state, and drei's CameraControls handles turning, zooming, preset views, and the smooth move to a fracture that S3.3 asks for.
- **Blender and glTF-Transform** prepare the Z-Anatomy model: removing bones, renaming meshes, cutting triangles, and exporting one compressed .glb file.
- **PDF.js** shows the original PDF unchanged, with highlight boxes drawn over it (S4.1, S4.2).
- **Python and FastAPI** run the report reader as a small web service.
- **pdfplumber** gets the text out of a PDF, along with where each word sits on the page.
- **medspaCy** finds a report's sections and handles "no fracture" and "possible fracture" wording. It is built for clinical text and works by rules; it does not generate text.
- **Supabase** provides accounts (with email confirmation and password reset), a Postgres database, and file storage. Its row-level security makes the database itself refuse a report to anyone but its owner. Its built-in email only reaches our own team, so confirmation and reset emails go through a free SMTP sender such as Resend.
- **Vercel, Render, and Supabase's hosted service** run the web app, the reader, and the data. All three have free tiers.
- **One GitHub repository** holds the web app, the reader, and bones.json (Key Decision 3), so both halves read the same bone list. GitHub Actions runs the tests and the model check on every push.
- **Vitest, pytest, and Playwright** test the web app, the reader, and the whole app in real browsers.

Architecture



The app has three parts. The web app runs in the patient's browser and does everything the patient sees: the screens, the PDF viewer, and the 3D skeleton, which it loads once as a static file along with bones.json. The report reader is a small Python service that takes a report and returns its text and proposed findings. Supabase holds accounts, the database of reports and findings, and the original files. Inside the web app, the Workspace Controller owns the confirmation gate, the one selected finding that the report pane and the skeleton both follow, and the timeline and comparison state.

1. **Adding.** The web app sends the report to the reader and gets back proposed findings.
2. **Checking.** The patient confirms or corrects the findings. Only then does the web app save the original file to Supabase Storage and the report and its findings to the database.
3. **Viewing.** The web app loads a report's findings from the database and draws them on the skeleton, beside the original report.

Key Decisions

1. **The report is never changed, and only its words place a marker.** The PDF or pasted text is stored exactly as added. Findings, highlights, and labels live in their own tables and are drawn on top of it (NFR4). Every finding points back to the exact report wording it came from, and a marker is drawn only for a finding the patient has confirmed or entered (S2.3). Scan images never move a marker (NFR5).

2. **The reader uses rules, not AI.** It finds fractures by matching the report against our own bone list and Appendix A's word lists, with medspaCy handling sections, "no fracture", and "possible fracture". The same report always produces the same findings, each one traces back to a rule, and no report is sent to an outside AI service. This is how we meet NFR6 and the instructor's guidance on LLMs. When a rule misses, the patient corrects it on the check screen (S2.3), and we add that case to the tests.
3. **One bone catalog drives the reader, the model, and the screens.** bones.json lists each of the 126 bones once: an id that is also the bone's mesh name in the model (radius_left), its medical and everyday names (S3.1), the words reports use for it ("radial", Appendix A.3), and its parts, with the point where each part's marker goes. The reader, the skeleton, and the correction menus all read this one file, so a bone can't be named one way in one place and another way somewhere else. A teammate also reviews this file thoroughly before release, checking every bone name against Radiopaedia.
4. **Left and right are settled once, in the model file.** Every mesh name ends in _left or _right, meaning the patient's side (Appendix A.4). In the model's coordinates the patient's left is the +X side. A check runs on every change to the model and fails if any _left bone sits on the wrong side, so the mirror-image mistake can't reach a marker.
5. **The reader keeps nothing.** It takes a report, returns its text and proposed findings, and forgets it. Only Supabase stores reports, so deletion happens in one place (NFR10), and the reader can be tested on its own, straight from the test set (NFR7).
6. **The findings format is fixed at the end of week 1.** The fields the reader returns (Appendix B.2) are agreed by both tracks in week 1. Milestone 3 draws markers from hand-written findings files in that format, so the skeleton and the reader are built at the same time and meet in Milestone 4 without rework.

Buckets of Work

The 3D Skeleton

The center of the product (F3). This bucket covers preparing the Z-Anatomy model (removing the 80 axial bones, naming the 126 that remain, and reducing it to 250,000 triangles), showing it in the browser, drawing a marker for each finding on the right bone, side, and part, and the controls for moving around it: turning, zooming, preset views, going to each fracture, and showing one bone on its own.

The Report Reader

This is the area with the most uncertainty, and the one NFR7 measures. It covers everything between a report coming in and a list of proposed findings going out: getting the text out of a PDF, finding the report's sections and exam date, and turning each fracture sentence into a finding with its bone, side, part, finger or toe, type words, certainty, and healing words (Appendix A). It also covers building and scoring the test set. Screens are not part of this bucket.

The Patient's App

The screens and storage that turn the reader and the skeleton into a product: accounts (F6); adding, dating, and deleting reports (F1); the check screen where the patient confirms and corrects findings (F2); the original report beside the model, linked in both directions (F4); and the recovery timeline (F5).

Milestones

Milestone 1: The Skeleton Is on Screen

This milestone gets the Z-Anatomy model into the browser, named, light enough to turn smoothly, and with left and right proven correct. Everything that draws a fracture depends on it.

How the model is prepared:

- **Removing the axial skeleton.** Open the Z-Anatomy model in Blender and delete the 80 axial bones (22 skull, 6 middle-ear ossicles, 1 hyoid, 26 vertebral column, 25 thoracic cage). Check that the 126 appendicular bones remain, each a separate mesh (Appendix A.3).
- **Naming.** Rename every mesh to its bones.json id: the bone, then the finger or toe number and position where needed, then the side. For example: humerus_left, metacarpal_5_right, proximal_phalanx_hand_1_left. The thumb and big toe have no middle phalanx (Appendix A.4).
- **Pose.** Check that the model stands in anatomical position with the palms facing forward. That is a property of the mesh, not of the camera or the lighting, so if it is wrong, it gets fixed in Blender.
- **Size.** Reduce the model to at most 250,000 triangles with Blender's Decimate modifier, keeping more detail in the hand and foot bones, since S3.2 zooms in to a single finger. Export one .glb compressed with glTF-Transform, aiming for under 5 MB.
- **The model check.** A script reads the .glb and fails if any mesh name is missing from bones.json, if any _left bone sits on the patient's right, or if the model has more than 250,000 triangles. It runs on every change to the model.
- **bones.json.** Written in week 1 (shape in Appendix B.4), since the mesh names here, the reader's bone words, and the marker points all come from it.
- **Setup.** Also in week 1: the repository, GitHub Actions, and a first deploy of the web app (Vercel) and the reader (Render), so later milestones are tested on the real hosting.

How the viewer works:

- The web app loads the .glb once and keeps it. React Three Fiber draws it, and drei's CameraControls handles turning, zooming, and returning to the starting view (S3.2).
- Buttons go straight to the front, back, left-side, and right-side views. The zoom limit lets a single finger fill the screen.
- Clicking a bone selects it and shows its medical and everyday names from bones.json (S3.1). A selected bone can be shown on its own, and the full skeleton brought back (S3.3).
- A notice on the viewer says it shows a standard skeleton, not the patient's own bones (NFR5). The only screen in this milestone is the skeleton viewer on its own, which later becomes the model half of the split view (A.5).

Unknowns:

- We haven't opened the Z-Anatomy file yet, so we don't know for sure that each of the 126 bones is its own separate mesh, that the skeleton is standing with its palms forward, or how many triangles it has. Some bones might be merged together (like all the finger bones as one object) or split into extra pieces.
- If Z-Anatomy doesn't work for us, our backups are BodyParts3D (the model Z-Anatomy is built from) or Open3DModel.
- How we decide: on day 1 of week 1, one of us opens the file in Blender, counts the bone meshes, and checks the pose and the triangle count. If it fails any of these and we can't fix it in Blender

within a day or two, we try the next model.

Proving it works:

- The model check passes.
- On the oldest laptop the team has with built-in graphics (made in 2020 or later), turning the full skeleton holds 30 frames per second (NFR3).
- Every control in S3.2 is tried in Chrome, Safari, Firefox, and Edge.

We'll know this milestone is complete when the skeleton loads in anatomical position, every S3.2 control works in all four browsers, the model check passes, and turning the model holds 30 frames per second on built-in graphics.

Estimate: 1 week.

Milestone 2: The Reader Finds Fractures

This milestone builds the report reader and measures it against real reports. It runs alongside Milestones 1 and 3, since nothing on screen is needed to test it.

How reading works: A report comes in as a PDF or pasted text and goes through these steps (S2.1, Appendix A):

- **Text.** pdfplumber pulls the text out of a PDF along with where each word sits on the page, which Milestone 5 uses for highlights. A PDF with no text in it, such as a scan, is turned away with a message until F8 exists.
- **Sections.** medspaCy's sectionizer finds Exam, History, Comparison, Technique, Findings, and Impression (Appendix A.1). Only Findings and Impression produce findings. A report with no headings is read as all findings.
- **Exam date.** Dates are taken from the Exam line and the header, never from Comparison. One date is filled in for the patient; no date, or several that disagree, means the patient enters it (S1.2).
- **Fractures.** A sentence with a fracture word, or a named fracture such as "Colles fracture" (Appendix A.5), becomes a finding when it names a bone. The bone comes from bones.json, including adjective forms like "radial". The side comes from the sentence or, failing that, from an Exam line that names exactly one side. The part and its bone ID (which includes the finger or toe number for hand and foot bones) come from the words next to the bone.
- **Negation and certainty.** medspaCy's ConText rules, loaded with the word lists in Appendix A.6, drop "no fracture of the ulna" and mark "suspected" or "cannot be excluded" as uncertain (S2.2). Healing words are recorded with the finding.
- **Duplicates.** The same fracture in Findings and Impression becomes one finding that keeps both pieces of wording (S2.1).
- **Can't place.** A finding on a bone we don't cover, with no side, or naming only a region like "wrist" is labeled with that reason and gets no marker (S2.4).

  What comes back is a list of proposed findings, each with its bone id, side, part, finger or toe, type words, certainty, healing words, status, and the exact report wording with its position (Appendix B.2).

The test set: We use de-identified real reports from team members and volunteers (OQ1).

- We collect at least 30 fracture reports from team members and volunteers, including follow-up reports, which OQ3 also needs. If we have fewer than 30 by the end of week 1, we write the rest

from Radiopaedia examples.

- Names, birth dates, record and accession numbers, and doctor and facility names are removed. Every date in one person's reports is shifted by the same number of days, so the gaps between visits stay the same.
- The set is kept in a private folder, never in the public repository or the deployed app.
- Two of us write each report's answer key separately and settle differences before scoring.
- Half the set is used while writing rules. The other half is held back and scored only at the end of this milestone and in week 6.

Unknowns:

- We picked medspaCy because it's built for clinical text, but we haven't actually tried it yet. We don't know how well it handles our reports, how long it will take us to learn, or if it fits in the 512 MB of memory Render's free tier gives us.
- The other options are spaCy's built-in Matcher with negspacy for the "no fracture" part, or plain regular expressions. Regex is the simplest, but then we write all the negation rules ourselves.
- How we decide: by day 3 of week 1, we run each option on the two Appendix B reports plus 5 reports from the test set and pick the one that gets the findings right with the least work. We also check how much memory it uses.
- We also don't know yet if the highlight positions will work. The reader returns where each finding's wording sits on the page (B.2), which means matching the sentence back to the words pdfplumber found, and then turning pdfplumber's page coordinates into boxes PDF.js can draw at any zoom level. If either step is off, the highlights land on the wrong words.
- Since the findings format is fixed at the end of week 1, we test this in week 1 instead of finding out in Milestone 5: we run the B.1 PDF through the reader, draw its boxes in PDF.js, and check that they cover the right words. If we can't get it working, the backup is to skip the coordinates and have the web app search for the wording in PDF.js's text layer instead.

Proving it works:

- Appendix B's two reports are the first tests. They must produce exactly the findings in B.1 and B.2, including B.2's exam date coming from the Exam line and not from Comparison.
- There is one test for each named fracture in Appendix A.5 and each word list in A.6, plus traps such as "no new fracture" and "negative for fracture".
- A scoring script runs the reader over the held-back half and reports NFR7's two numbers and the time taken per report (NFR1).

We'll know this milestone is complete when Appendix B passes exactly, every A.5 and A.6 test passes, and on the held-back half at least 90% of fractures are found with the right bone and side, no more than 5% of findings name a bone or side the report doesn't, and each report takes under 10 seconds.
Estimate: 2 weeks, running through weeks 1 and 2.

Milestone 3: Fractures Are Marked on the Skeleton

This milestone draws findings on the model. It uses findings files written by hand from Appendix B, in the format fixed in week 1 (Key Decision 6), so it doesn't wait for the reader.

How marking works:

- Each placed finding gets one marker on its bone and side. When the finding names a part, the

marker sits on that part's point from bones.json; when it doesn't, the whole bone is highlighted (S3.1).

- A script finds each long bone's two ends and the middle of its shaft. Named parts, such as the femoral neck, the malleoli, and the radial styloid, are placed by hand in Blender. We check every point by eye.
- Markers are attached to their bone's mesh, so they stay on it from every angle and at every zoom level (S3.2).
- Certain and uncertain findings use different marker shapes and labels ("Possible fracture"), never color alone (S3.1, NFR12).
- The findings list sits beside the model. Choosing a finding moves the camera so that fracture is centered, and next and previous buttons step through them (S3.3).
- Findings that can't be placed are listed with their reason and get no marker (S2.4).

Proving it works:

- B.1's finding produces exactly one marker, on radius_left at its distal end. Seen from the front, it is on the viewer's right. An automated screenshot test checks this, which catches a mirrored model from end to end.
- A test file with one finding on each of the 126 bones produces 126 markers, each attached to the mesh its finding names.
- In a grayscale screenshot, the uncertain markers can still be told apart from the certain ones.

We'll know this milestone is complete when B.1's fracture shows on the left distal radius, all 126 test markers land on the right meshes, and choosing any finding centers it in the view.

Estimate: 1 week.

Milestone 4: Adding and Checking a Report

This is where the reader, the skeleton, and accounts first meet: a patient signs in, adds a report, checks what the reader found, and sees it on the model.

How it works:

- **Accounts.** Supabase handles sign-up with a confirmed email, sign-in, and password reset by email (S6.1, S6.2). An account can't store reports until its email is confirmed. Signing out leaves nothing of the patient's on that computer.
- **Adding.** The patient uploads a PDF (up to 10 MB and 10 pages) or pastes text (S1.1). The web app sends it to the reader, then asks for the exam date, filled in when the reader found one (S1.2). The exam date can be changed later too. The reader's free hosting can take up to a minute to wake, so the web app wakes it as soon as the patient signs in.
- **Checking.** The check screen lists every proposed finding with its report wording. The patient can remove a finding, change its bone, side, or part, or add one the reader missed using menus built from bones.json. Anything they add or change is labeled as entered by them (S2.3). If the reader finds no fractures, the screen says so (S2.2).
- **Saving.** Only when the patient confirms does the web app save the original file to Supabase Storage and the report and findings to the database. A report moves from Added to Reading to Unconfirmed to Confirmed, and only a confirmed report reaches the model (Appendix B.3).
- **Privacy.** Row-level security on every table lets a row be read or changed only by the account that owns it (NFR8). The reader only accepts requests from our own site with a signed-in user's

token, and never logs report text.

- **Deleting.** Deleting a report asks for confirmation, then removes its findings and file too (S1.3). Deleting an account requires signing in again; a small Supabase function then deletes the account's files and the account, and the database removes everything that belonged to it (S6.3).

The screens in this milestone are Registration/Login, Adding a Report, and Verify Findings (A.2, A.3, and A.4).

Proving it works:

- With two test accounts, account B tries to read, change, and delete account A's reports and files, both through the app and directly through Supabase's API. Every attempt must fail.
- A report added on one computer is there after signing in on another, and nothing is left on the first computer after signing out.
- An 11 MB PDF, an 11-page PDF, and a Word file are each turned away, and nothing is saved.
- Sign-up works with an email address that is not on our team.
- After an account is deleted, a database query finds no rows and no files left for it (NFR10), and the same email can sign up again (S6.3).

We'll know this milestone is complete when a new user can sign up, add Appendix B.1 both as a PDF and as pasted text, correct a finding, see its marker, find everything again on a second computer, and every attempt in the cross-account test fails.

Estimate: 1 week.

Milestone 5: Report and Model Together

This milestone puts the original report beside the model and links the two, so a patient can go from a sentence to a bone and back (F4).

How it works:

- PDF.js shows every page of the original PDF in a scrolling pane beside the model; pasted text is shown as text. The model can take the full width, and the report can be brought back (S4.1).
- Each finding's wording is highlighted with boxes drawn over the page from the word positions the reader returned. The file itself is never changed (NFR4).
- Clicking a highlight selects that finding and moves the camera to its marker. Clicking a marker scrolls the report to its wording. Both panes follow one selected finding, held by the workspace controller (S4.2).
- A finding the patient entered has no wording to show and is labeled as entered by them.

The screen in this milestone is the split view (A.5).

Unknowns:

- Whether the highlight boxes line up with the right words in the PDF. We test this in week 1 (see the Milestone 2 unknowns), so it should already be settled by the time we get here.
- If the week 1 test failed and we switched to the backup, the highlights come from searching PDF.js's own text layer for each finding's wording instead of from the reader's boxes. The rest of this milestone stays the same.

Proving it works:

- An automated browser test on B.1 clicks the highlighted sentence and checks that the marker is centered, then clicks the marker and checks that the sentence is in view.
- We look over the highlights on every PDF in the test set to make sure each box covers the right

words.

- On the deployed app, with the reader already woken at sign-in, findings are ready within 10 seconds of adding a 10-page PDF (NFR1), and the model with every marker appears within 5 seconds of confirming them (NFR2).

We'll know this milestone is complete when the report and model show together for both PDFs and pasted text, both directions of linking work for every finding in Appendix B, and the NFR1 and NFR2 times pass on the deployed app.

Estimate: 1 week.

Milestone 6: Recovery Timeline

This milestone lets a patient follow an injury across visits (F5).

How it works:

- Reports are listed by exam date, oldest first, and reports with the same date keep the order they were added. Each shows its date and the bones it involves, and moving to the next or previous report changes the markers (S5.1).
- Comparing two reports draws both sets of markers on one skeleton, so they are always seen from the same angle. Each finding is marked by shape and label as in both reports, only the earlier one, or only the later one (S5.2).
- Whether two findings are the same injury is decided by one function, sameInjury, which uses same bone and same side until OQ3 is settled. Changing the rule later is a single edit.
- Healing status comes only from the report's healing words, shown with the matching label from Appendix A.6 and the report's own wording. A fracture with no healing words shows no status, and the time between visits is never used (S5.3).

The screen in this milestone is Recovery Timeline and Report Comparison (A.6).

Proving it works:

- Appendix B's two reports, added in either order, list B.1 first. Comparing them shows the radius fracture in both, with a healing status and "interval callus formation" only on B.2.
- A report added later with an earlier exam date takes its place by exam date, and two reports with the same date keep the order they were added.
- The follow-up reports in the test set show where the same-bone, same-side rule goes wrong, which is the evidence OQ3 is waiting on.

We'll know this milestone is complete when Appendix B's reports appear in exam order however they were added, compare correctly, and show a healing status only where the report states one.

Estimate: 1 week.

Timeline

Weeks 1 and 2 run two tracks at once, with two of us on the skeleton and two on the reader. The findings format is fixed at the end of week 1, and features freeze at the end of week 5, so week 6 is only testing and the demo build. If we finish early, we start on the P2 features in the order Section 4 of the requirements lists them, beginning with F7. M3 needs M1 and the findings format, M4 needs M2 and M3, and M5 and M6 need M4. Week 6 is our only buffer: if a milestone slips into it, the week 6 testing shrinks, not the P1 features.

Week	Work
1	M1: The Skeleton Is on Screen. M2 starts: The Reader Finds Fractures. Findings format fixed at the end of the week. Setup and bones.json done.
2	M2 finishes. M3: Fractures Are Marked on the Skeleton.
3	M4: Adding and Checking a Report
4	M5: Report and Model Together
5	M6: Recovery Timeline. Feature freeze at the end of the week.
6	Full testing: the held-back reports for NFR7, a first-use test with 5 newcomers for NFR11 (on accounts we create beforehand, since guest mode is P2), a legibility check on a 13-inch laptop (NFR12), and all four browsers. Demo build.

4. Risks

Factors that could derail the plan, ordered roughly by severity.

1. Parser Accuracy (NFR7): Diverse clinical phrasing may cause missed or incorrect findings.

Mitigation: Build a 30+ report test set early, score changes against held-back cases, and add every case the reader misses as a new test.

Technical/Dependency Risk

2. Viewport Performance: Complex 3D mesh rendering could cause lag on lower-end devices.

Mitigation: Remove axial bones, cap the appendicular model at 250,000 triangles, compress to GLB (<5 MB), and enforce automated CI build checks.

Technical Risk

3. Anatomical & Coordinate Correctness: Mirror-image confusion may place left-side findings on the viewer's left instead of the patient's left.

Mitigation: Standardize bone mesh naming suffix (`_left`/`_right`), validate coordinate bounds in CI, and run end-to-end screenshot tests.

Technical Risk

4. Schedule Constraints: A tight 6-week timeline creates bottleneck risks across dependent milestones.

Mitigation: Freeze findings data contracts in Week 1, lock P1 MVP scope, freeze features by Week 5, and restrict P2 features to post-MVP.

Scheduling Risk

5. Team Experience: Little to no experience with Three.js/React Three Fiber, Blender, medspaCy, and Supabase row-level security.

Mitigation: The week 1 unknowns checks (Milestones 1 and 2) are where we learn each tool, with one owner per tool.

Personnel Risk

6. Free-Tier Limits: Supabase's built-in email only reaches our own team, and Render's free reader can take up to a minute to wake. Either could break sign-up (S6.1) or the 10-second target (NFR1).

Mitigation: A free SMTP sender for email, and waking the reader at sign-in.

Dependency/Cost Risk

7. Outside Dependencies: The plan relies on the Z-Anatomy model being usable (licensed CC BY-SA 4.0) and on volunteers sending reports in week 1.

Mitigation: Backup models (Milestone 1 unknowns), Radiopaedia examples for missing reports, and a credit for Z-Anatomy in the app.

Dependency Risk

8. Test Set Privacy: The test set uses real reports, even with personal details removed.

Mitigation: It stays in a private folder, never in the repository or the deployed app, and demos use only the fictional reports from Requirements Appendix B.

Dependency/Technical Risk

5. References

- 3D Views for Medical Reports – Software Pitch
- 3D Views for Medical Reports – Requirements Document V2
- Radiopaedia – Public radiology reference for fracture terms and report wording (Requirements Section 9, Appendix A)
- Z-Anatomy and Repository – Reference for the 126 appendicular 3D skeletal model (Requirements Appendix C.1); interactive 3D skeletal convention. Licensed CC BY-SA 4.0, so the app credits it and our edited model keeps the same license.
- Merck Manual Skeletal System – Reference for anatomical naming convention; interactive 3D skeletal convention.
- Technical Documents:
    - Three.js: https://threejs.org/docs/
    - React Three Fiber: https://github.com/pmndrs/react-three-fiber
    - drei: https://github.com/pmndrs/drei
    - glTF-Transform: https://gltf-transform.dev/
    - Blender manual: https://docs.blender.org/manual/en/latest/
    - PDF.js: https://mozilla.github.io/pdf.js/
    - FastAPI: https://fastapi.tiangolo.com/
    - pdfplumber: https://github.com/jsvine/pdfplumber
    - medspaCy: https://github.com/medspacy/medspacy
    - Supabase: https://supabase.com/docs
    - Vitest: https://vitest.dev/
    - Playwright: https://playwright.dev/

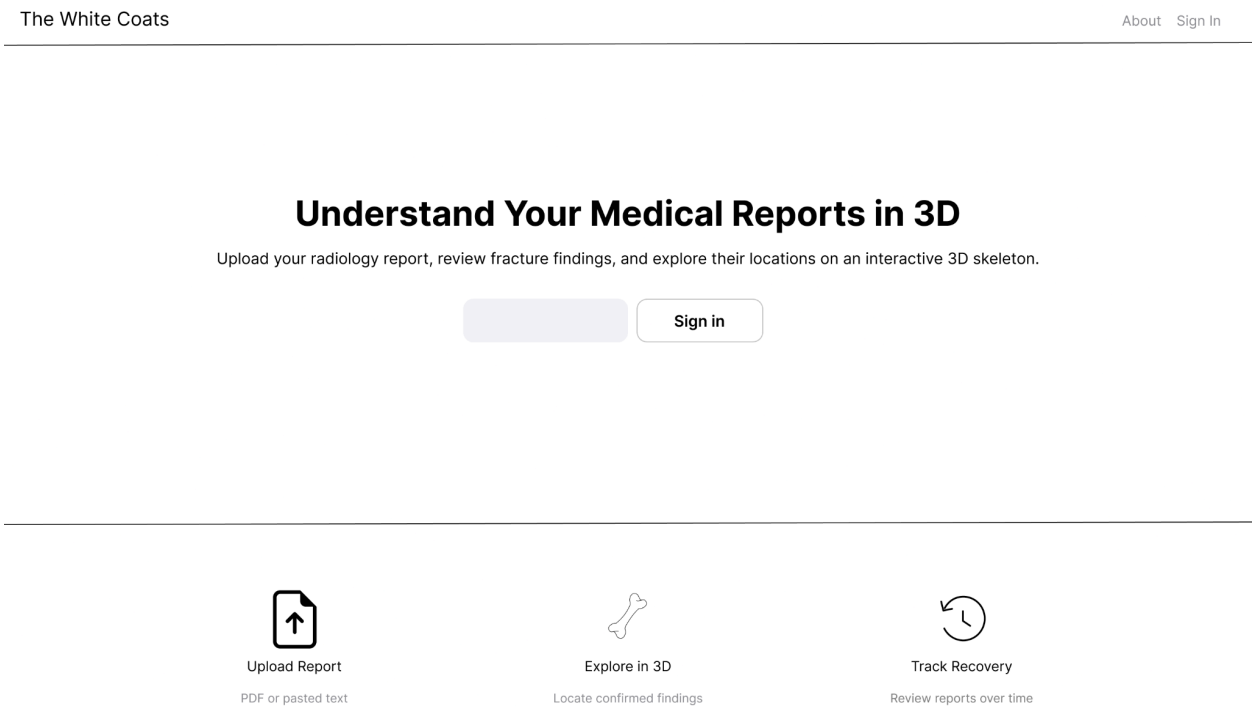
6. Appendices

Appendix A: UI Wireframes

Six MVP screens, in the order a new patient meets them. Each heading below says what its screen shows and which stories it covers.

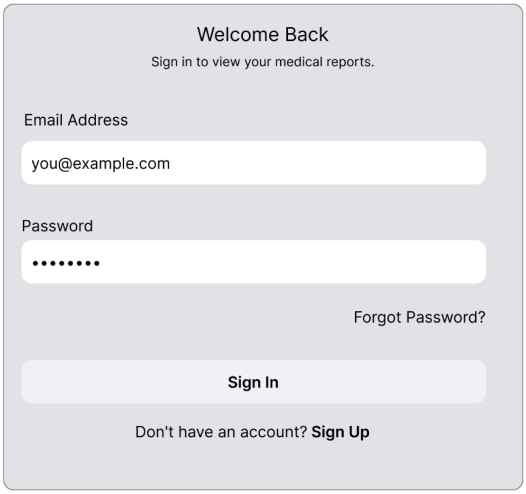
A.1: Splash Screen, Onboarding

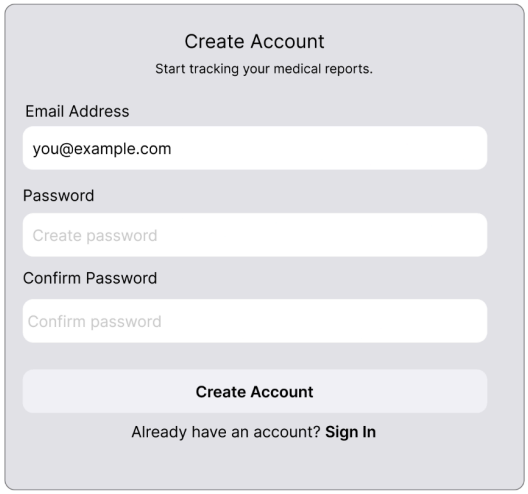
What the app does and does not do (a standard skeleton, no diagnosis), then Sign in or Create account (NFR5, NFR11).



A.2: Registration/Login

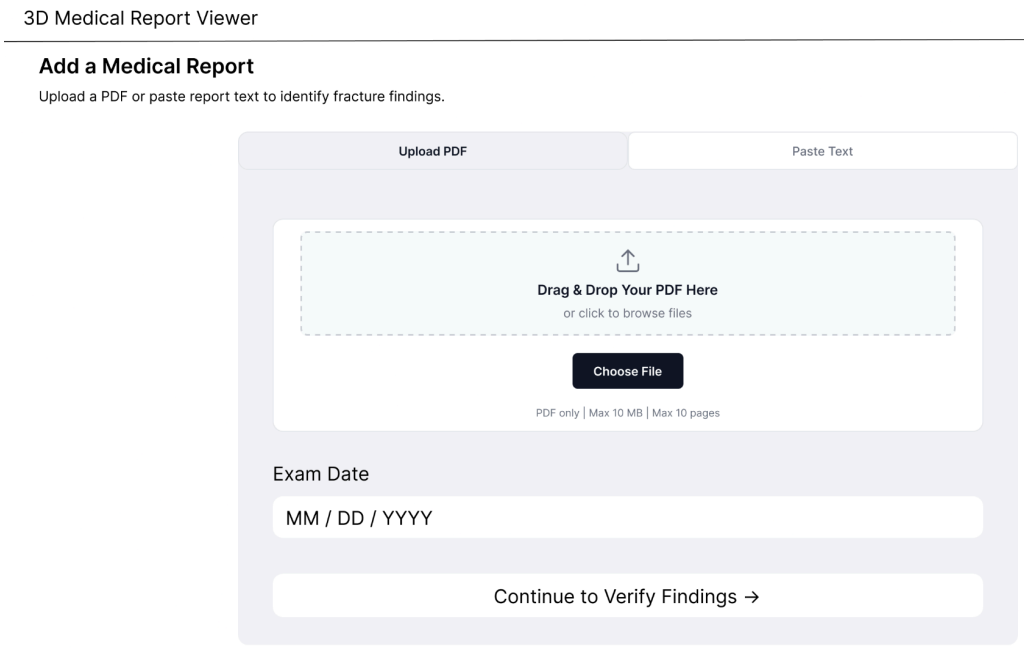
Email and password only, email confirmation, and password reset by email (S6.1, S6.2). A returning patient lands on the timeline (A.6).





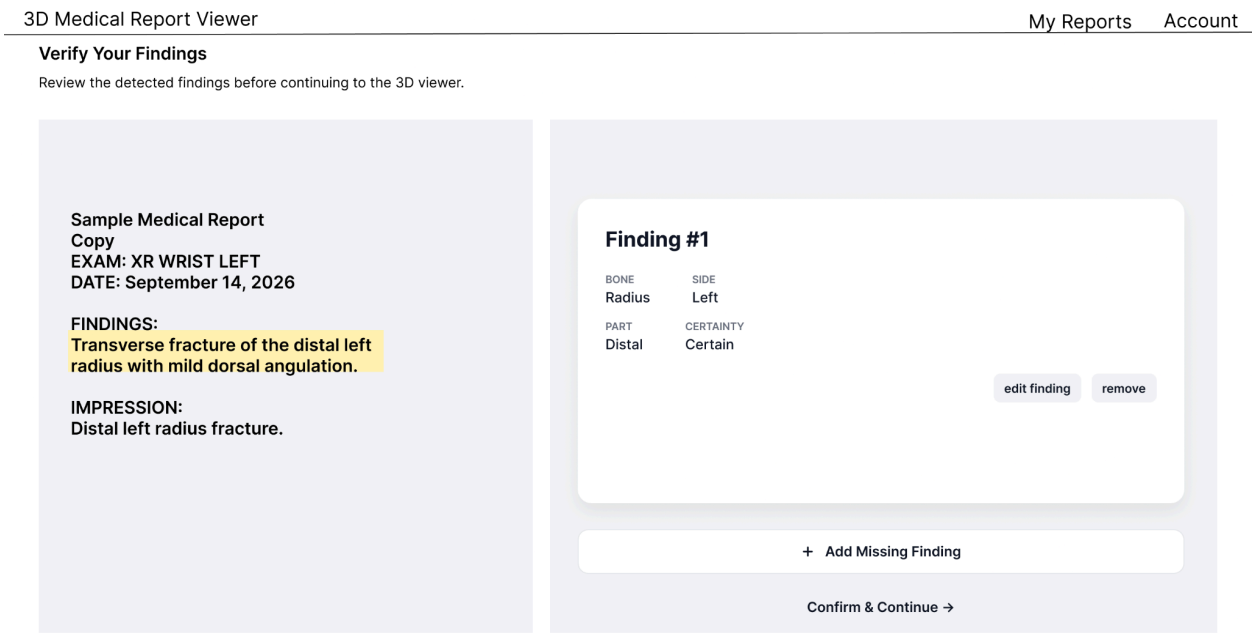
A.3: Adding a Report

Upload a PDF or paste text; the exam date is filled in when the reader finds one, or asked for (S1.1, S1.2).



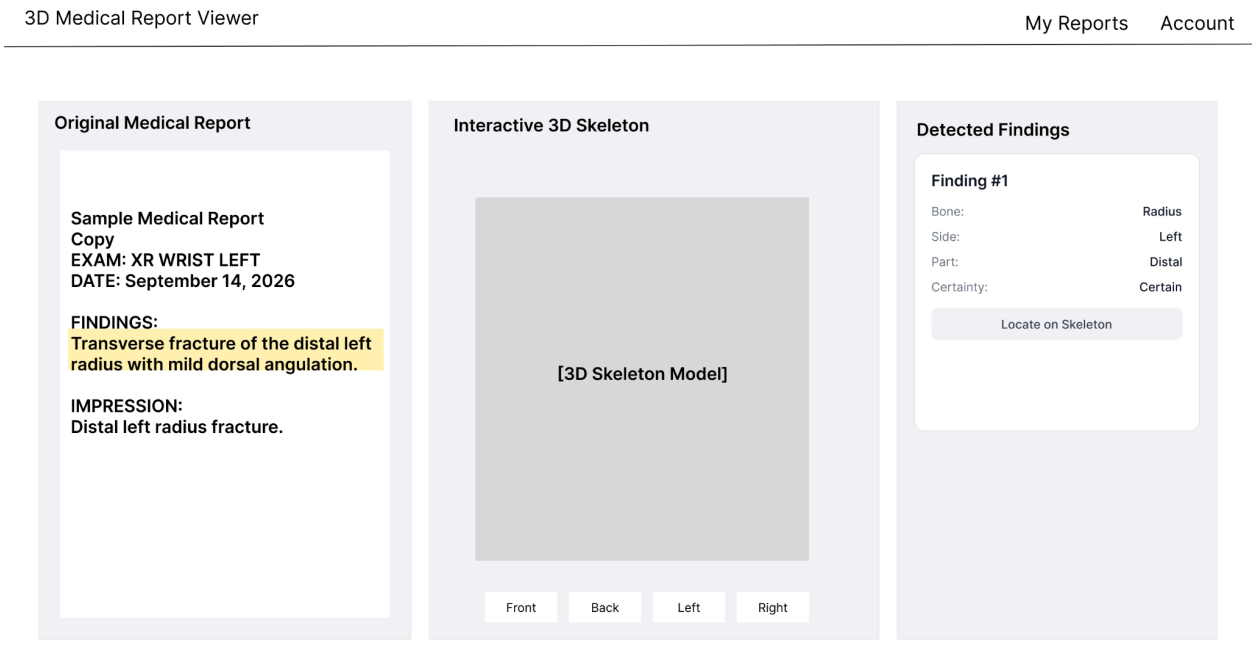
A.4: Verify Findings

Every proposed finding with its report wording; remove, edit, or add findings; findings the app can't place are listed with their reason (S2.1 to S2.4).



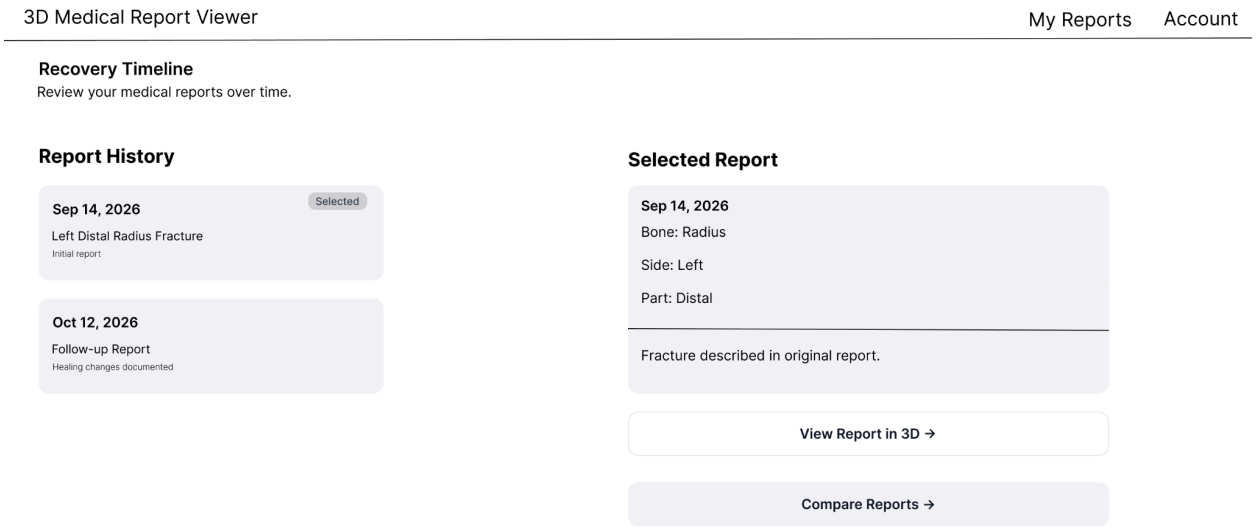
A.5: Loading Screen, Split-View of Report + 3D Skeleton

The original report on one side and the skeleton on the other, with the findings list, preset view buttons, and the standard-skeleton notice (S3.1 to S3.3, S4.1, S4.2). In the front view, a left-side fracture appears on the viewer's right.



A.6: Recovery Timeline, Report Comparison

Reports by exam date, previous and next, and a picker for comparing two reports on one skeleton (S5.1 to S5.3).



Appendix B: Data Schemas and Contracts

B.1: Disk Storage

Supabase holds two tables plus the original files in Storage, saved at {user_id}/{report_id}.pdf so Storage's own access rules limit each file to its owner. Accounts live in Supabase Auth, which keeps only the email address and a password hash (NFR8). Row-level security on both tables limits every row to its owner.

Reports

Column	Type	Notes
id	uuid	Primary key
user_id	uuid	Owner; row-level security checks it (NFR8); deleted with the account (S6.3)
source	text	PDF or pasted
file_path	text	Original PDF in Storage; empty for pasted text
raw_text	text	Text as extracted or pasted, never edited (NFR4)
exam_date	date	Required, not in the future (S1.2)
exam_date_source	text	report or patient
added_at	timestamptz	Keeps the added order for same-day exams (S5.1)

Findings

Column	Type	Notes
id	uuid	Primary key
report_id	uuid	Deleted with its report (S1.3)
bone	text	bones.json bone without the side, e.g. radius or metacarpal_5; empty if not stated
side	text	left, right, or empty; the mesh is bone + _ + side
part	text[]	Every part named, e.g., distal and metaphysis; empty means the whole bone
type_words	text[]	Fracture-type and alignment words as written
certainty	text	certain or uncertain (S2.2)
healing_words	text[]	Exact healing words; empty if none (S5.3)

status	text	placed, outside_coverage, side_not_stated, or bone_not_stated (S2.4)
wording	jsonb	Each piece of report wording with its position: page and boxes for a PDF, character range for text (S4.2)
entered_by_patient	boolean	Shown as entered by me (S2.3)

B.2: Report Reader Contract

The web app calls the reader once per report. This contract is fixed at the end of week 1 (Key Decision 6); changes after that need both tracks to agree.

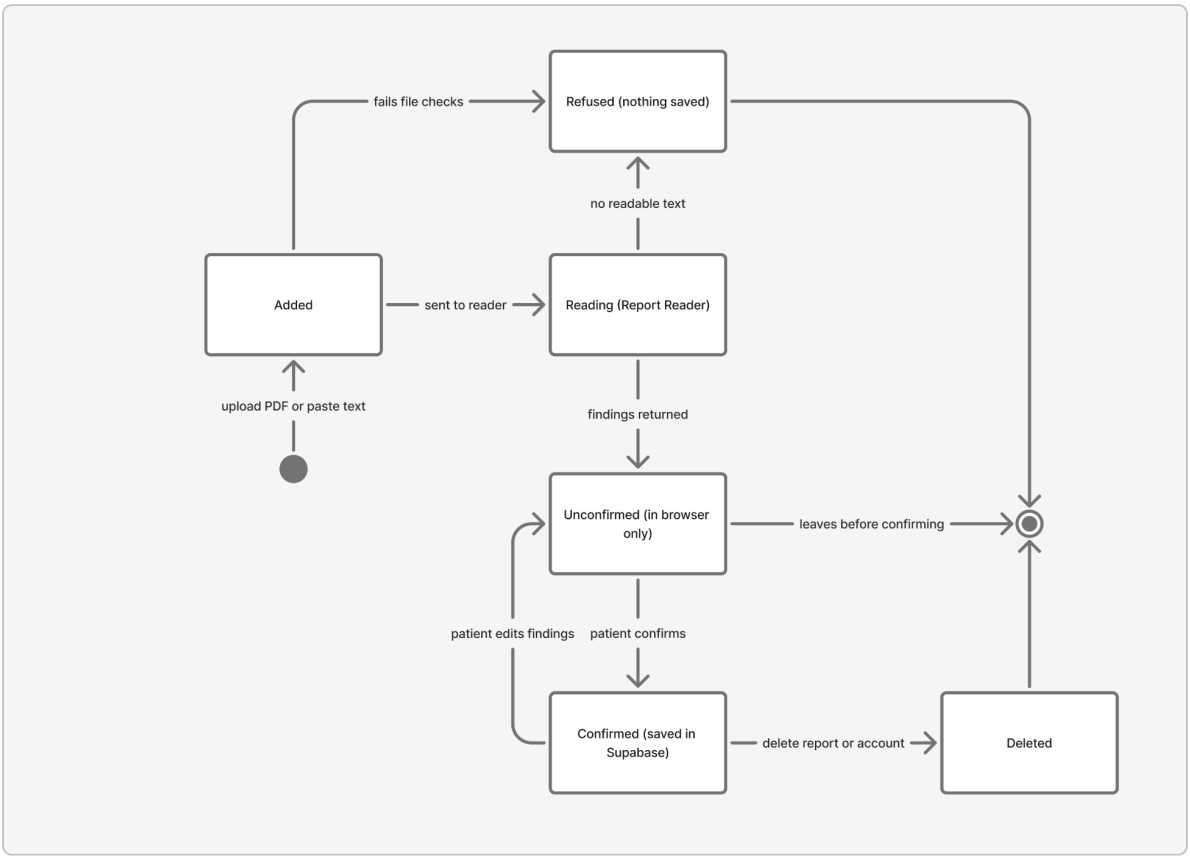
- **Request:** POST /read with a PDF file or pasted text.
- **Response:** the extracted text, the exam date (or the candidate dates when there is not exactly one), and a list of proposed findings with the B.1 findings fields except id and report_id.
- **Refusals:** not a PDF (not_pdf), over 10 MB (too_large), over 10 pages (too_many_pages), or no readable text (no_text), each with its own message (S1.1). Nothing is stored either way (Key Decision 5).

Example response for the Requirements Appendix B.1 report (boxes are [x0, top, x1, bottom] in PDF points):

```
{
  "text": "EXAM: XR WRIST LEFT, 3 VIEWS ...",
  "exam_date": "2026-09-14", "candidate_dates": [],
  "findings": [{
    "bone": "radius", "side": "left", "part": ["distal", "metaphysis"],
    "type_words": ["transverse", "mild dorsal angulation"],
    "certainty": "certain", "healing_words": [], "status": "placed",
    "entered_by_patient": false,
    "wording": [{ "page": 1, "boxes": [[72, 210, 540, 224]], "text": "Mildly angulated
distal left radius fracture." },
                { "page": 1, "boxes": [[72, 300, 410, 314]], "text": "Mildly angulated
distal left radius fracture." }]
  }]
}
```

B.3: Report Processing & Lifecycle State Machine

B.3 Report Processing & Lifecycle State Machine



B.4: bones.json

One entry per bone, 126 in all. The id is also the bone's mesh name in the model (Key Decision 3), and a finding with no part highlights the whole bone.

```
{
  "id": "radius_left", "bone": "radius", "side": "left",
  "medical": "radius", "everyday": "forearm bone, thumb side",
  "words": ["radius", "radial"],
  "parts": { "distal": [0.31, 0.92, 0.04], "shaft": [0.30, 1.05, 0.03] }
}
```